Більше, ніж пройдені тести: огляд специфікації лише за інтерфейсом скорочує тихі збої локального агента-програміста на Gemma

Олександр Пилипенко

Незалежний дослідник у сфері штучного інтелекту та машинного навчання

Аспірант Державного торговельно-економічного університету, Київ, Україна

У співпраці з кафедрою інженерії програмного забезпечення та кібербезпеки

ORCID: 0009-0004-9610-9242

O.Pylypenko@knute.edu.ua

2026-10-21

Анотація

Агенти-програмісти зазвичай зупиняються після проходження доступних тестів, навіть якщо ті не охоплюють сформульованих вимог. Ми досліджуємо цей збій у схемі «хмарний архітектор / локальний виконавець», де `gemma4:12b` виправляє невеликі модулі Python на ноутбучному GPU. Хмарний рецензент отримує лише сигнатури, docstring і поля типів даних, ніколи тіла функцій, та повертає контрактні тести або чекліст. Окрема прихована перевірка оцінює отриманий код. На 50 екземплярах задач і 150 парних запусках із виправленим вимірюванням група контрактних тестів зменшила тихі хибні успіхи з 37 до 4 і підвищила верифіковане виконання з 66,0% до 87,3%; на рівні задач тихі збої зменшилися у 12 випадках і не зросли в жодному ($p = 0{,}0005$, post hoc). Це об'єднані задачі, частково використані під час розробки, а не 50 незалежних нових специфікацій. У зареєстрованій кампанії на 25 нових задачах обидві форми огляду підвищили верифіковане виконання з 58/75 до 71/75 і зменшили тихі хибні успіхи з 9/75 до 1/75. Чекліст дав ту саму сукупну частку верифікованого виконання, що й контрактні тести; еквівалентність не перевірялася. З урахуванням одноразової генерації на задачу, повторно використаної для трьох seed, час кампанії зріс на 11,8% для чекліста і на 53,7% для тестів. Зареєстрований тест на рівні задач не досяг значущості ($p = 0{,}125$); приріст контрактних тестів зосереджений у багатофайлових задачах. Розвідувальна абляція вказує на важливість docstring. Результати підтримують застосування огляду специфікації в цих обмежених умовах; його причинні складові та ефективність на дефектах реальних репозиторіїв залишаються відкритими питаннями.

Ключові слова: агенти-програмісти; великі мовні моделі; тестування за специфікацією; тестові оракули; тихі збої; Gemma; Gemini; Google Antigravity; попередня реєстрація.

Незалежність від Google. Дослідження оцінює архітектурну схему, яку Google описав публічно; воно не узгоджувалося з Google, не спонсорувалося і не рецензувалося ним, і жодних тверджень про продукти Google, окрім процитованого оголошення, не робиться. Усі експерименти виконано на обладнанні та облікових записах автора.

1 Вступ

Пройдений набір тестів — звичайний критерій зупинки для автономного агента-програміста. Водночас він слабкий: коли тести покривають лише частину специфікації,

агент може задовольнити тести й порушити решту. Дослідження бенчмарків показали, що недостатні тести завищують коректність згенерованого коду: розширення тестів HumanEval знизило pass@k на 19,3–28,9% для 26 моделей [10]. Для агента, що працює без нагляду, така помилка гірша за видиму: агент повідомляє про успіх, і ніщо далі не заперечує.

Ми натрапили на цей збій, будуючи гібридний оркестратор, у якому хмарний «архітектор» планує, а локальна модель редагує код, щоб вихідний код залишався на машині. Саме таку схему Google запропонував для локальних моделей в Antigravity SDK: хмарна модель Gemini планує лише за назвами файлів і описами, а локальні екземпляри Gemma виконують роботу на пристрої [9]. З локальним виконавцем на 12B параметрів головним збоєм на задачах із насиченою специфікацією була не нездатність редагувати код чи йти за планом: виконавець проходив видимі тести й мовчки пропускав правила, сформульовані лише в docstring. Збільшення ліміту генерації, режим міркування при повторній спробі, підказки хмари після невдалої спроби та одноетапна адаптація обв'язки цього не змінили (підрозділ 6.7).

Ця стаття ставить вузьке питання: **чи може хмарна модель, що читає лише специфікацію і пише контрактні тести або чекліст правил, зменшити тихі збої локального агента-програміста так, щоб реалізація не залишала машину, і якою ціною?** Наші внески:

1. Вимірювання тихих хибних успіхів локального агента-програміста на задачах із насиченою специфікацією з незалежною прихованою перевіркою як еталоном.

2. Механізм з обмеженим розкриттям, у якому хмара бачить лише сигнатури, docstring і поля типів даних, і докази того, що повнота інтерфейсу визначає, допомагають контрактні тести чи шкодять.

3. Попередньо зареєстрована кампанія з контрольним чеклістом, що перевіряє, чи додає виконуваність огляду вимірювану користь, і процедура побудови задач (жеребкування видимих правил; баги локальної моделі), що зменшує свободу автора обирати умови на користь методу.

4. Оцінювання, що розділяє висновки на рівні запусків і на рівні задач, звітує негативні результати для п'яти альтернативних втручань і перевіряє згенеровані тести на надмірну суворість і мутаційне покриття без участі агента.

Ми не стверджуємо, що генерація тестів мовними моделями є новою (розділ 2), ані що ефект поширюється за межі досліджуваного класу задач (розділ 8).

2 Пов'язані роботи

Недостатні тести. EvalPlus розширив вхідні дані тестів HumanEval приблизно у 80 разів за допомогою генерації мовними моделями та мутацій і виявив, що багато розв'язків, раніше визнаних правильними, хибні [10]. Наші приховані перевірки відіграють роль сильніших тестів; нас цікавить власний критерій зупинки агента, а не оцінювання бенчмарка.

Тести, згенеровані мовними моделями. CodeT генерує тести тією самою моделлю, що пише кандидатні розв'язки, і впорядковує розв'язки за узгодженістю з цими тестами [3]. AgentCoder відокремлює агента-програміста від агента-тестувальника, який пише тести за вимогами [5]. TestGen-LLM у Meta покращує наявні тестові класи, написані людьми, і залишає лише кандидатів, що проходять фільтри вимірюваного покращення; у впровадженні 75% тестів скомпілювалися, 57% надійно проходили, а 25% підвищили покриття [1]. Найближча до нашої постановки робота — генерація тестів за специфікацією в Google: спершу виводяться передумови, післяумови й невизначена поведінка, а потім за ними пишуться

тести; це підвищило виявлення помилок на виробничих багах на 9,8 відсоткового пункту ($p = 0{,}0352$) [12]. Наш механізм відрізняється напрямком потоку інформації: тести пише *інша, віддалена* модель, яка ніколи не бачить тіл функцій, а тести слугують критерієм приймання для *локального* агента, а не продуктом-набором тестів.

Агенти, що обходять тести. ImpossibleBench змінює тести так, що вони суперечать специфікації природною мовою, і вимірює, як часто агенти все одно їх проходять [13]. Наш тип збою доповнювальний: агент не маніпулює тестами; він зупиняється, щойно видимі тести пройдено, і лишає нереалізованими правила, яких тести не перевіряють.

Адаптація обв'язки. Harness[2] адаптує обв'язку агента — промпти, процедури, інструменти — без зміни ваг моделі [4]. Одноетапна адаптація нашого виконавця в цьому стилі не покращила результат на відкладених задачах (підрозділ 6.7).

Тестові оракули, самозворотний зв'язок і зовнішня валідність. Проблема оракула полягає у визначенні очікуваної поведінки навіть за автоматичної генерації входів; специфікації й контракти є усталеними джерелами оракулів [2]. Наші приховані тести — операційний оракул, а не повна специфікація. Self-Refine повідомляє про покращення завдяки ітеративному самозворотному зв'язку [11], а Huang та співавтори виявляють обмеження внутрішньої самокорекції міркувань без зовнішнього зворотного зв'язку [6]; різні умови застерігають від узагальнення нашого негативного результату SPEC. SWE-bench досліджує реальні дефекти репозиторіїв [8], а LiveCodeBench збирає нові задачі для зменшення ризику контамінації [7]. Тут ми не оцінюємо жоден із цих бенчмарків. Приховані правила контролюють доступ до оцінювання в експерименті, але не доводять відсутності контамінації навчання чи перенесення на реальні дефекти.

3 Постановка задачі та метрики

Задача складається з модуля з дефектом, функції якого мають специфікації в docstring, файлу видимих модульних тестів і прихованої перевірки, яка перевіряє правила docstring, пропущені видимими тестами. Для одного запуску агента нехай $V \in \{0, 1\}$ означає, що власні видимі тести задачі проходять після запуску, $H \in \{0, 1\}$ — що проходить прихована перевірка, а $S \in \{0, 1\}$ — що агент повідомляє про успіх.

- **Верифіковане виконання** (основна метрика): $V = 1 \wedge H = 1$.
- **Тихий хибний успіх:** $S = 1 \wedge (V = 0 \vee H = 0)$; на практиці майже завжди $V = 1$, $H = 0$.
- **Вартість:** час запуску агента, локальні й хмарні токени, а також одноразова генерація огляду для задачі.

V і H оцінює бенчмарк після запуску лише на власних тестових модулях задачі (згенеровані контрактні тести виключено); прихована перевірка ніколи не показується жодній із моделей.

Тут *верифіковане виконання* означає успіх за визначеною процедурою оцінювання, а не доведення повної функціональної коректності. Прихована перевірка сама пропускає деякі правила docstring (підрозділ 6.5).

4 Система

Програмне забезпечення тепер поширюється як *CloudEdge AI Orchestrator*; у записах експериментів збережено історичну назву й простір імен Python. Початковий тег `paper-i-preprint-2026-10-21` залишається на коміті `8e9ba05ebbbcaf7953a8f31287d935f01fa2d6be`.

Локальний агент. `gemma4:12b` в Ollama 0.33.3 (дайджест моделі `4eb23ef187e2…`) з власним викликом інструментів над набором інструментів робочої теки (читання, запис і латання файлів; запуск команди перевірки), без режиму міркування, з контекстом 16 384 токени, лімітом генерації 2 048 токенів, який для однієї повторної спроби підвищується до 4 096, коли виклик інструмента обрізано, підказками етапу, що називають наступний крок і тест, який не проходить, та циклом самовиправлення до двох спроб із дедлайном 300 с на спробу. Обладнання: один ноутбучний GPU NVIDIA GeForce RTX 4060 (8 ГБ) і 31 ГБ RAM; модель частково вивантажується на CPU.

Інтерфейсне навантаження. Для файлів, які може змінювати підзадача, і нетестових вхідних модулів, якими вони користуються, обхід AST видає назви класів, сигнатури функцій, docstring і анотовані поля класів (назви й типи; значення за замовчуванням відкинуто). Тіла функцій та решту коду пропущено, а навантаження перед виходом із машини проходить перевірник приватності оркестратора.

Автор контрактних тестів. Хмарна модель отримує навантаження та інструкцію написати один модуль `unittest`, що перевіряє кожне правило docstring — результати для типових і граничних входів та `ValueError` для кожного описаного виду некоректного входу — і не перевіряти нічого, чого docstring не стверджує. Отриманий модуль статично перевіряється (імпорти лише `unittest` і модулів, що тестуються; без `open`, `exec`, `eval`, `getattr` тощо; без dunder-атрибутів; наявний підклас `TestCase`) і, якщо коректний, записується поруч із власними тестами задачі як `test_spec_contract.py`, де його знаходить звичайна команда перевірки агента. Тести генеруються один раз для задачі й повторно використовуються для всіх seed. Хмарний бекенд — Antigravity CLI (`agy` 1.4.3), запущений у порожній тимчасовій теці з інструкцією не використовувати інструменти. У прогонах R1–R5 використовувалася модель CLI за замовчуванням, яку не було зафіксовано; у кампаніях A і B модель задано явно.

Автор чекліста. Для контрольної групи та сама хмарна модель отримує те саме навантаження й повертає пронумерований перелік правил docstring без коду; перелік додається до опису задачі, який читає агент.

Межа приватності. Назви, типи й docstring самі можуть розкривати інформацію; тому ми описуємо механізм як *обмежене розкриття специфікації*, а не як формальну гарантію приватності.

5 Дизайн експериментів

Усі запуски в усіх групах використовують однаковий цикл агента, дедлайн і ліміт спроб, по три seed (303, 404, 505) на задачу, з чергуванням порядку груп для кожного seed. Задачі — невеликі задачі з виправлення коду на Python (один-два модулі), у яких видимі тести покривають лише частину docstring; кожну задачу перевірено заздалегідь: початковий код і часткові виправлення не проходять, а еталонні виправлення проходять і видимі, і приховані перевірки.

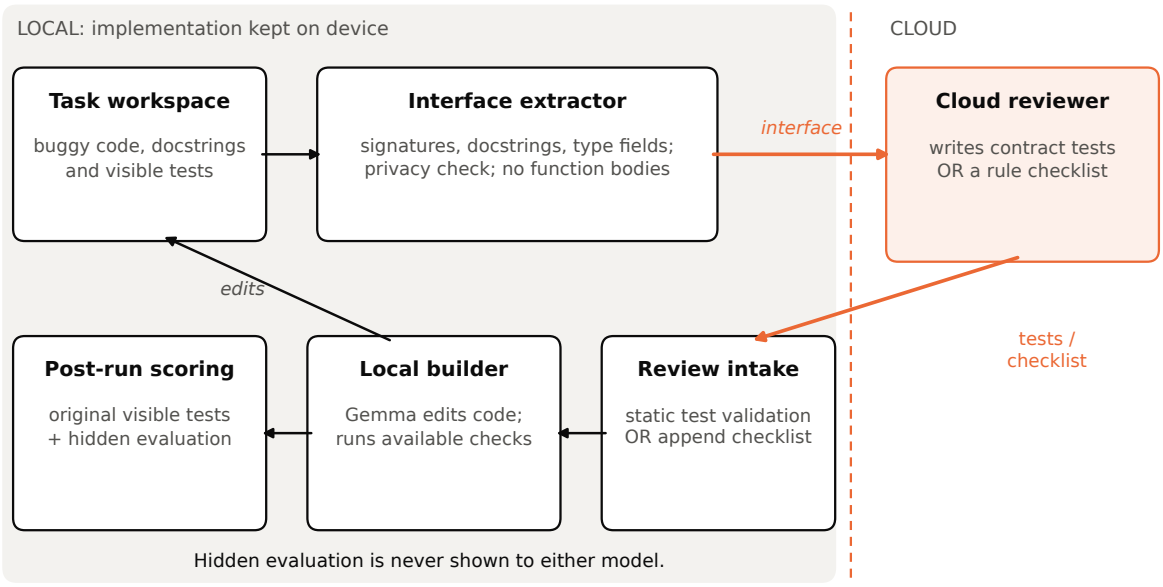


Рис. 1: Потік інформації. Межу з хмарою перетинає лише інтерфейс (сигнатури, docstring, поля типів), а назад повертається лише статично перевірений модуль модульних тестів (або чекліст правил); агент редагує код локально, доки не пройде видимі й контрактні тести. Рішення про приймання ухвалюється після запуску за власними видимими тестами задачі та прихованою перевіркою, якої не бачить жодна модель.

Докази зібрано у п'яти прогонах на 19 задачах (таблиця 1) та у двох зареєстрованих кампаніях: A на 25 нових специфікаціях і B на шести повторно використаних. Прогони R1–R3 були розвідувальними; між R3 і R4 знайдено й виправлено помилку вимірювання (підрозділ 6.6); R4 було попередньо зареєстровано; R5 повторює порівняння R3 з виправленим вимірюванням.

Базова політика — агент із підказками етапу й адаптивним лімітом генерації. *SPEC* після проходження видимих тестів дає агентові сигнатури й docstring його задачі та вимагає одноразової самоперевірки кожного правила. *SPECTESTS* встановлює хмарні контрактні тести до запуску (v1: лише власний файл функцій; v2: також поля типів даних і залежні модулі). *ASSIST* після невдалої спроби просить у хмари покрокові вказівки. Між R3 і R4 для обох груп внесено зміну: одиничне застрягання в межах спроби тепер дає нову спробу, а не завершує підзадачу.

Таблиця 1: Прогони на перших 19 задачах.

Прогін	Коміт	Задачі	Групи	Мета
R1	`cc938ca`	H6, H9–H13 (6)	базова, SPEC, SPECTESTS, ASSIST	розвідувальне порівняння
R2	`24fbe35`	H1–H5, H7, H8 (7)	базова, SPECTESTS v1	шкода на розв'язаних задачах
R3	`750647e`	H1–H13 (13)	базова, SPECTESTS v2	порівняння в одній сесії
R4	`7a369ac`	H14–H19 (6, нові)	базова, SPECTESTS v2	зареєстроване підтвердження
R5	`7a369ac/2c59e38`	H1–H13 (13)	базова, SPECTESTS v2	повтор із виправленим вимірюванням

Попередня реєстрація R4. До будь-якого запуску на H14–H19 основну кінцеву точку (верифіковане виконання), групи, seed, ліміт спроб, парування за (задача, seed) і точний двосторонній знаковий тест було записано в журнал прогонів проєкту. Журнал — локальний файл поза системою контролю версій, тому ця реєстрація не має незалежної часової мітки (розділ 8). Перша спроба R4 упала на зависанні й використала невиправлене вимірювання; її відкинуто й повторено без змін, окрім виправлення вимірювання.

Статистика. Порівняння на рівні запусків використовують точні двосторонні знакові тести на розбіжних парах (задача, seed). Оскільки запуски однієї задачі не є незалежними, ми також аналізуємо різницю частки верифікованого виконання для кожної задачі (контрактні тести мінус базова політика, за трьома seed): знаковий тест на рівні задач, точний перестановочний тест зі зміною знаків і кластерний бутстреп за задачами (20 000 повторень). Аналіз на рівні задач додано після зовнішньої рецензії; щодо R4 він є post hoc.

Підтверджувальні кампанії A і B. Щоб відповісти на загрози з розділ 8 — задачі й баги, написані автором уже після виявлення типу збою, замало нових задач, незафіксована хмарна модель і відсутність контролю на те, що хмара просто читає специфікацію, — ще дві кампанії було зареєстровано публічними комітами до їх запуску (додаток A).

- *Задачі.* Двадцять п'ять нових задач із виправлення коду (P1–P25: 8 однофайлових, 10 багатофайлових, 7 зі свідомо стислими, недовизначеними docstring) з предметних областей, яких раніше не було (гроші, текст, дати, одиниці, розклад, статистика, конфігурація, матриці, обмеження частоти, форматування). Кожен docstring формулює пронумеровані правила, і кожне правило має тестові випадки. **Які правила покривають видимі тести, автор не обирає:** для кожного правила вирішує жереб із фіксованим seed ($p = 0{,}5$) з єдиним обмеженням: щонайменше одне видиме правило має не проходити на коді з дефектом. Прихована перевірка перевіряє всі правила. Вміст задач заморожено за SHA-256 для кожної задачі.

- *Баги.* У кампанії A код із дефектом пише автор. У кампанії B його пише сама локальна модель: для кожної специфікації `gemma4:12b` реалізує модуль за скелетом із docstring (seed генерації 1–5, температура 0,8, тести не показуються); оригінальні docstring відновлюються, щоб дефект був лише в тілах функцій, а першою задачею стає перша реалізація, яка порушує правило. Специфікації, які модель реалізує правильно на всіх seed, виключаються й підраховуються.

- *Групи.* Базова політика; контрактні тести; та **контрольна група «чекліст»**. Контроль порівнює форми огляду за тієї самої хмарної моделі й інтерфейсу; він не ізолює потужність моделі чи незалежність. Кампанія A додає розвідувальну абляцію того, скільки інтерфейсу бачить хмара (лише сигнатури; плюс docstring; повний).

- *Налаштування.* Як у R4–R5, із зафіксованою хмарною моделлю (Gemini 4 Argon у кампанії A; Gemini 3.8 Flash у кампанії B за зареєстрованою поправкою) та окремим записом часу одноразової генерації для кожної задачі. Seed 303, 404, 505. Основний тест: точний двосторонній знаковий тест за задачами для частки верифікованого виконання (контрактні тести проти базової); вторинні: тихі хибні успіхи, контрактні тести проти чекліста, результати за категоріями, вартість.

6 Результати

6.1 Розвідувальне порівняння (R1)

На шести задачах із насиченою специфікацією верифіковане виконання становило 7/18 для базової політики, 5/18 з локальною самоперевіркою (SPEC), 16/18 з контрактними тестами

Таблиця 2: Аналіз R4 і R5 на рівні задач (одиниця — задача; різниця частки верифікованого виконання за трьома seed, контрактні тести мінус базова політика).

Набір	Краще/гірше/рівно	Знаковий тест p	Зміна знаків p	Середній приріст (95% CI бутстрепу)
Нові H14–H19 (R4)	3/0/3	0,25	0,25	+0,39 (+0,06, +0,72)
H1–H13 (R5)	4/1/8	0,38	0,13	+0,23 (+0,03, +0,46)
Усі 19 задач	7/1/11	0,070	0,023	+0,28 (+0,09, +0,49)

(SPECTESTS) і 8/18 з підказками після невдачі (ASSIST); SPECTESTS проти базової дали 10 перемог і 1 поразку ($p = 0{,}012$ на рівні запусків). Запусків, що пройшли видимі тести, але не пройшли прихованої перевірки, було 11 для базової, 10 для SPEC, 0 для SPECTESTS і 10 для ASSIST. R1 оцінював групу SPECTESTS за суворішим критерієм, ніж базову (підрозділ 6.6).

6.2 Повнота інтерфейсу вирішує між допомогою і шкодою (R2, R3)

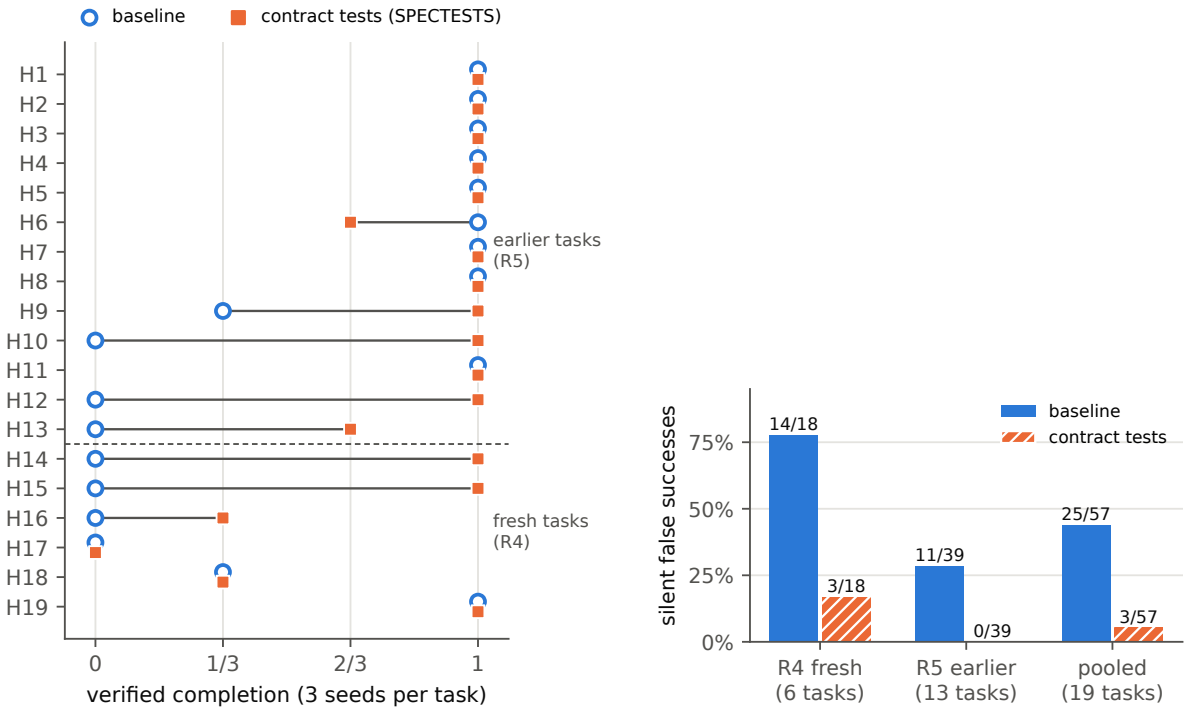
На семи задачах, які базова політика вже розв'язує, перша версія контрактних тестів (v1), написана лише за власним файлом функцій, знизила верифіковане виконання з 21/21 до 15/21 (0 перемог, 6 поразок, $p = 0{,}031$). П'ять поразок припало на H1 і H3, де типи даних визначено в інших модулях; перевірка згенерованих модулів на еталонних реалізаціях показала 19 з 20 хибних тестів для H1 (неіснуюча назва поля) і 11 з 12 для H3 (пропущений аргумент конструктора). Коли в навантаження додано поля типів і залежні модулі (v2), кожен згенерований модуль для 13 задач пройшов на своїй еталонній реалізації, а розв'язані задачі втратили 2 з 21 запуску (незначущо); на чотирьох задачах, де базова здебільшого зупинялася на видимих тестах, верифіковане виконання зросло з 2/12 до 10/12 (8 перемог, 0 поразок, $p = 0{,}0078$).

6.3 Зареєстрований прогін на нових задачах (R4)

На шести раніше не використаних задачах верифіковане виконання становило 4/18 для базової і 11/18 з контрактними тестами (8 перемог, 1 поразка, 9 нічиїх; $p = 0{,}039$ на рівні запусків), а тихих хибних успіхів було 14/18 і 3/18. На рівні задач покращилися три задачі (H14 0/3→3/3, H15 0/3→3/3, H16 0/3→1/3), погіршень не було, три не змінилися: знаковий тест $p = 0{,}25$, середній приріст +0,39 (95% CI кластерного бутстрепу 0,06–0,72). Усі три тихі збої, що залишилися, припали на H17; єдині порушені приховані правила стосуються порожніх пар (`"a=1;;b=2"`, `"a=1;"`), а таке прочитання docstring спірне; оцінка H17 без цих двох випадків дала 7/18 проти 14/18 з тими самими 8 перемогами й 1 поразкою. Усі шість згенерованих модулів пройшли на незалежних еталонних реалізаціях.

6.4 Усі 19 задач із виправленим вимірюванням (R4 + R5)

R5 повторив порівняння R3 на H1–H13 з виправленим вимірюванням: 28/39 проти 37/39 (10 перемог, 1 поразка, $p = 0{,}012$ на рівні запусків), тихі хибні успіхи 11/39 проти 0/39, а на дев'яти задачах, які базова розв'язує 3/3, верифіковано 26 з 27 запусків SPECTESTS (одна поразка на H6). Разом для R4 і R5 (19 задач, однаковий рушій і перевірки) верифіковане виконання становило 32/57 проти 48/57, а тихі хибні успіхи 25/57 проти 3/57. Таблиця 2 і рисунок 2а показують картину на рівні задач, а рисунок 2б — тихі хибні успіхи. Приріст зосереджено в семи задачах, правила яких сформульовано лише в docstring (H9, H10, H12, H13, H14, H15, H16); одинадцять задач не змінилися, а одна (H6) втратила один запуск із трьох.



(а) Верифіковане виконання за задачами.

(б) Тихі хибні успіхи.

Рис. 2: R4 і R5. (a) Верифіковане виконання для кожної задачі (три seed) для базової політики (порожні кола) і з контрактними тестами (заповнені квадрати); задачі нижче пунктирної лінії — нові задачі зареєстрованого прогону R4. (b) Запуски, у яких агент повідомив про успіх, а незалежна перевірка не пройшла.

6.5 Якість згенерованих тестів

Згенеровані тести досліджено без участі агента. **Надмірна суворість.** Сорок сім різних реалізацій, які в запусках *без* контрактних тестів пройшли і видимі тести, і приховану перевірку, відтворено із записаних diff (геші початкового стану звірено) і прогнано на контрактних модулях v2: не пройшли 6 із 91 пари «реалізація–модуль», усі на H4 (від'ємна вага має викликати помилку) і H9 (знак + або не-ASCII символ у prerelease має бути відхилено). У кожному випадку docstring формулює правило, а реалізація його порушує; прихована перевірка цього правила не перевіряє. Жодного контрактного тесту, що вимагав би поведінки, якої docstring не формулює, не знайдено. **Мутаційне покриття.** Із 448 одноточкових AST-мутантів еталонних реалізацій (19 задач) прихована перевірка вбила 333; із них видимі тести вбили 205 (61,6%), а контрактні модулі v2 — усіх (469 з 469 оцінених пар «мутант–модуль»; один мутант може оцінюватися кількома модулями, тому цей знаменник відрізняється від 333 унікальних мутантів; рисунок 3). Ще 84 мутанти пережили приховану перевірку, але були вбиті контрактними тестами; кожен із них під час перегляду порушує правило docstring. Тридцять один мутант пережив усі перевірки. Для H14–H16 еталоном слугувала реалізація з прогону з контрактними тестами, що пройшла приховану перевірку, бо жоден запуск базової політики цих задач не розв'язав. Наслідок: приховані перевірки слабші за специфікації, тож «верифіковано» завищує коректність в обох групах; оскільки контрактні тести забезпечують пропущені правила, це упередження грає на користь базової політики.

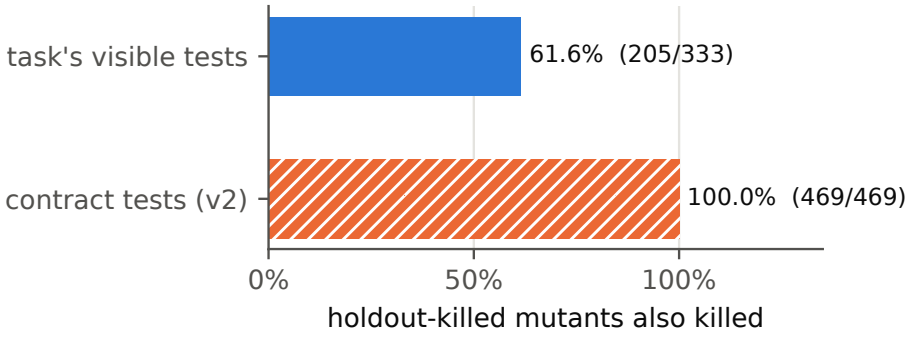


Рис. 3: Із одноточкових мутантів, яких убиває прихована перевірка, частка, яку також убивають видимі тести задачі та контрактні тести v2 (для контрактних тестів — пари «мутант–модуль»).

6.6 Виправлення вимірювання і вартість

У R1–R3 незалежна видима перевірка для групи SPECTESTS запускала також згенеровані тести, а прихована перевірка виконувалася лише за умови, що ця перевірка пройшла; тому запуски SPECTESTS оцінювалися суворіше, ніж запуски базової політики, що могло лише знизити їхні оцінки. R4, R5 і обидві кампанії використовують виправлені перевірки (лише власні тести задачі, прихована перевірка завжди виконується, зависання вважається невдачею).

Контрактні тести подовжують запуски: у R4 сумарний час становив 929 с проти 4 014 с, а локальні токени — 256 017 проти 1 171 194, плюс 91 958 хмарних токенів на шість викликів генерації; у R5 — 3 092 с проти 5 273 с і 614 904 проти 1 378 211 локальних токенів, плюс приблизно 264 000 хмарних токенів на 13 задач. Базова політика швидка почасти тому, що зупиняється, щойно пройдено слабкі видимі тести. Ці значення часу не включають одноразову хмарну генерацію для задачі, яку в R1–R5 не вимірювали; співвідношення (1,7–4,3×) є нижніми оцінками повної вартості. Кампанії записують час генерації окремо.

6.7 Втручання, що не допомогли

- **Самоперевірка тією самою моделлю** (SPEC, R1): 5/18 проти 7/18; самоперевірки агента перевіряли лише ту поведінку, яку він уже реалізував.
- **Підказки хмари після невдачі** (ASSIST, R1): 8/18; підказка вмикається після невдалої спроби, а перша спроба зазвичай проходила видимі тести, тож вона спрацювала один раз на 18 запусків.
- **Ескалація ліміту генерації, міркування при повторі та допомога хмари** у двоспробовій драбині: на H6–H8 відмінності дала одна задача, і їх віднесено до розкиду між запусками.
- **Одноетапна адаптація обв'язки** у стилі Harness[2] [4]: на відкладених задачах обв'язка за замовчуванням верифікувала 7/12 запусків, а адаптовані — 6/12, із довшим сумарним часом (829 с проти 601 с); стандартне правило відбору та правило з перевіркою зробили однакові вибори на кожній задачі, тож експеримент їх не розрізняє.

6.8 Кампанія A: 25 нових задач із жеребкуванням видимих правил (зареєстрована)

Виконано всі 225 основних запусків (3 групи × 25 задач × 3 seed). Для кожної задачі та форми огляду один етап генерації (до двох спроб) використовував Gemini 4 Argon; отримані

Таблиця 3: Кампанія A, усі запуски (25 задач × 3 seed). Генерація — одноразовий виклик хмари для задачі.

Група	Верифіковано	Тихі хибні успіхи	Час запусків	Генерація	Хмарні токени
Базова політика	58/75	9/75	6 675 s	—	0
Контрактні тести	71/75	1/75	9 630 s	626 s	627 787
Чекліст (контроль)	71/75	1/75	6 983 s	477 s	550 901

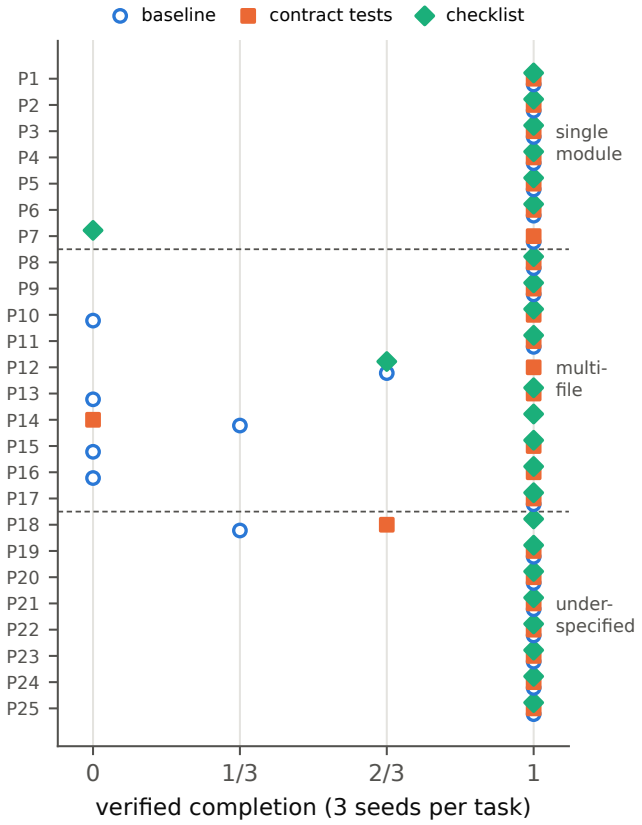


Рис. 4: Кампанія A: верифіковане виконання для кожної задачі (три seed) для базової політики (порожні кола), контрактних тестів (квадрати) і контрольного чекліста (ромби). Пунктирні лінії розділяють однофайлові, багатофайлові й недовизначені задачі.

артефакти повторно використано для всіх seed. Валідні модулі тестів установлено для 24 задач; надано 25 чеклістів (таблиця 3, рисунок 4).

Основну гіпотезу (H1) не підтверджено. На рівні задач контрактні тести покращили 6 задач і погіршили 1 (18 без змін): знаковий тест $p = 0{,}125$, перестановочний тест $p = 0{,}0625$, середній приріст частки верифікованого виконання +0,17 (95% CI кластерного бутстрепу 0,04–0,33). Напрям збігається з попередніми прогонами, але зареєстрований тест не досягає $\alpha = 0{,}05$. Тихих хибних успіхів (H2) стало менше в 3 задачах і більше в жодній ($p = 0{,}25$).

Додатний бутстреп-інтервал для середнього й незначущий знаковий тест не суперечать одне одному. Знаковий тест використовує лише напрям ненульових ефектів задач, а бутстреп оцінює їхню середню величину за емпіричною вибіркою. Основним правилом рішення залишається зареєстрований знаковий тест; інтервал його не замінює.

Переваги над чеклістом не виявлено (H3). Чекліст дав ту саму сукупну частку верифікованого виконання, що й контрактні тести (71/75), і ту саму кількість тихих хибних успіхів (1/75). На окремих задачах результати різнилися: тести проти чекліста — 2 краще, 2 гірше, 21 рівно ($p = 1{,}0$). Межу еквівалентності або не меншої ефективності не реєстрували й не перевіряли; незначуща різниця не доводить жодного з цих тверджень. Виконуваність не була необхідною для отримання цих сукупних результатів, але порівняння не ізолює причину користі огляду.

Де ефект. Для контрактних тестів проти базової політики всі зміни припали на 10 багатофайлових задач (6 краще, 1 гірше; середнє +0,43). Базова політика вже проходила кожен запуск 8 однофайлових і 7 недовизначених задач. Через цей ефект стелі первинний знаковий тест має лише сім розбіжних задач. Чекліст окремо погіршив однофайлову CSV-задачу P7 (нижче). Контрактні тести не погіршили жодної задачі без прихованих порушень.

Невдачі двох хмарних груп. P14 з контрактними тестами не вдалася 3/3: агент вичерпав спроби, а (правильні) контрактні тести так і не пройшли; P7 з чеклістом не вдалася 3/3 так само, після того як чекліст розписав правила лапок у CSV; обидва випадки — явні невдачі. Для P18 обидва згенеровані модулі тестів відхилив статичний перевірник (вони використовували `__dict__`), тож група з контрактними тестами на цій задачі працювала без тестів, як і передбачала реєстрація.

Надійність генерації. Валідний модуль установлено на 24/25 задачах (96%); одна задача (P18, 4%) залишилася без модуля після відхилення обох дозволених спроб генерації. Це частота на рівні задач, а не окремих відповідей. Три запуски P18 збережено в аналізі за призначеною групою (intention-to-treat). Збій хмарного бекенду зупиняє прогін; статичне відхилення є результатом роботи методу.

Якість згенерованих тестів. На еталонних виправленнях не пройшли 4 тести в 3 з 24 встановлених модулів: три надмірно суворі — на недовизначеній P20 вони вимагають `ValueError` для порожнього списку й дробового результату, а на P12 тест вимагає, щоб делегування до `statcore.mean` було видно для підміни (monkey-patching), що залежить від способу імпорту, а не від поведінки; четвертий (P10) очікує дробове `0.0` там, де docstring каже 0.0, а еталон повертає ціле 0. Жоден із цих тестів не спричинив невдалого запуску.

Скільки інтерфейсу потрібно хмарі (розвідувальна абляція, seed 303). Лише із сигнатурами контрактні тести залишили верифіковане виконання на рівні базової політики (19/25, 2 тихі; базова 20/25 на цьому seed); додавання docstring дало 25/25 (0 тихих); повний інтерфейс із полями типів дав 23/25 (1 тихий). На цих задачах цінність міститься в docstring; поля типів тут нічого не додали, тоді як на H1 і H3, типи даних яких визначено в окремих модулях, їх відсутність породила неможливі тести (підрозділ 6.2). Один seed на рівень, тож різниця в один-два запуски перебуває в межах розкиду.

Вартість з урахуванням генерації. Сума часу запусків і одноразової генерації на задачу становила 6 674,85 с для базової політики, 10 255,96 с для контрактних тестів і 7 459,95 с для чекліста: накладні витрати кампанії 53,7% і 11,8% відповідно. Сам час запусків зріс на 44,3% і 4,6%. Кожен згенерований артефакт повторно використано для трьох seed, тож це амортизована вартість кампанії, а не виміряні витрати одноразового застосування. Вона не включає налаштування програм, аналіз і людську працю та не вимірює енергію чи гроші. У R1–R5 генерацію не вимірювали; їхні часові співвідношення не є повними витратами.

Таблиця 4: Кампанія B, усі запуски (6 задач × 3 seed; описово).

Група	Верифіковано	Тихі хибні успіхи	Час запусків	Генерація	Хмарні токени
Базова політика	9/18	3/18	5 971 s	—	0
Контрактні тести	12/18	0/18	6 472 s	494 s	168 740
Чекліст (контроль)	12/18	2/18	6 112 s	126 s	105 621

6.9 Кампанія B: баги, написані локальною моделлю (зареєстрована; описова)

Побудова. Коли `gemma4:12b` попросили реалізувати кожну з 25 специфікацій кампанії A за скелетом із docstring, вона написала правильний модуль на всіх п'яти seed генерації для 19 специфікацій (104 генерації; одну відхилено як неповну). Природний баг дали лише шість специфікацій (N1, N7, N8, N9, N13, N18), що менше за зареєстрований поріг у десять, тож кампанію B звітовано описово, як і зареєстровано. У чотирьох із шести жереб зробив усі порушені правила видимими; лише N7 і N18 мають правила, порушені в прихованій формі. За пронумерованих правил у docstring локальна модель рідко лишає сформульоване правило нереалізованим, коли пише код з нуля; тихі збої попередніх задач виникали, коли вона виправляла код за неповними тестами.

Результати. Виконано всі 54 запуски з Gemini 3.8 Flash як хмарною моделлю (таблиця 4). На N18 контрактні тести верифікувалися в 3 з 3 запусків, чекліст — в 1 з 3, базова — в жодному; на N13 допоміг лише чекліст (2 з 3); N7, згенерований парсер якої зациклюється на деяких входах, не розв'язала жодна група. Усі шість згенерованих модулів тестів пройшли на еталонних виправленнях. На шести задачах ці результати описові; вони вказують у тому самому напрямку, що й кампанія A, причому контрактні тести — єдина група без жодного тихого хибного успіху.

6.10 Об'єднані докази (post hoc)

Об'єднання всіх запусків із виправленим вимірюванням — R4, R5 і кампаній A та B, 50 задач, контрактні тести проти базової політики на тій самій задачі й seed — дає загальний розмір ефекту (таблиця 5). Цей аналіз визначено після прогонів і подано як описову підтримку, а не як зареєстрований тест. На рівні запусків у кампанії A контрактні тести виправили 15 запусків, у яких базова зазнала невдачі, і зламали 2 ($p = 0{,}002$), а чекліст виправив 17 і зламав 4 ($p = 0{,}007$); обидва усунули 9 з 10 розбіжних тихих хибних успіхів ($p = 0{,}02$). Запуски з різними seed однієї задачі не незалежні, тож значення на рівні запусків завищують силу доказів для нових задач; значення на рівні задач є консервативнішим підсумком.

Чутливість до спільних специфікацій (post hoc). 50 екземплярів охоплюють 19 попередніх задач, 25 задач кампанії A і 6 варіантів специфікацій A у кампанії B. Об'єднання кожної Pk з її варіантом Nk дає 44 групи за походженням специфікації. Усередині групи об'єднуємо запуски; у знаковому тесті кожна група має однакову вагу: верифіковане виконання покращується у 13 групах і погіршується у 2 ($p = 0{,}00739$); тихі хибні успіхи зменшуються у 12 і не зростають у жодній ($p = 0{,}000488$). Ця перевірка не встановлює незалежності всіх груп, а враховує одну відому залежність. Об'єднаний аналіз залишається розвідувальним. Скрипт: `scripts/analysis/publication.py`.

Таблиця 5: Об'єднані запуски з виправленим вимірюванням (50 задач × 3 seed; post hoc).

Показник	Базова	Контрактні тести	Задач краще/гірше (знаковий тест p)
Тихі хибні успіхи	37/150	**4/150**	12/0 (0,0005)
Верифіковане виконання	99/150 (66%)	**131/150** (87%)	14/2 (0,0042)

7 Обговорення

Прогалина в покритті критерію зупинки. Спостережувані збої узгоджуються з недостатнім покриттям критерію зупинки: агент часто зупинявся після проходження видимих тестів і залишав нереалізовані правила docstring. Альтернативні втручання не показали надійного приросту за перевірених задач і бюджетів (підрозділ 6.7); це не свідчить про їхню неефективність в інших умовах.

Що встановлює контроль. Чекліст і виконувані тести дали однакові сукупні результати кампанії A, а виміряний час кампанії був меншим для чекліста (11,8% проти 53,7% накладних витрат із генерацією). Це узгоджується з користю явного подання вимог специфікації. Дослідження не розділяє внески незалежності рецензента, потужності моделі, структурованого подання, додаткової явної інформації й моменту її надання. Зокрема, SPEC був локальною самоперевіркою після проходження видимих тестів, а не локальним чеклістом, створеним до виправлення. Його невдача не доводить необхідності незалежності й потужності моделі окремо. Майбутнє зареєстроване порівняння має зафіксувати момент і інтерфейс та порівняти локальний чекліст до виправлення, хмарний чекліст і безпосереднє подання початкових правил. Тести залишаються виконуваними артефактами аудиту, але додаткової переваги виконання над чеклістом тут не виявлено; еквівалентність не встановлено.

Розкриття та надмірна суворість. Абляція на одному seed вказує, що docstring містили корисну інформацію понад сигнатури. Поля типів були необхідні для деяких попередніх задач, але в кампанії A не дали виміряного приросту. Неповні, неоднозначні або залежні від реалізації вимоги можуть породжувати надмірно суворі тести, зокрема на P12 поза категорією недовизначених задач. Жоден спостережуваний невдалий запуск не приписано розглянутим надмірним вимогам; це не доводить їхньої безпечності в інших умовах.

Межі й наступна перевірка. Приріст контрактних тестів у кампанії A зосереджений у багатофайлових задачах; простіші задачі часто мали стельовий baseline. Це мотивує майбутню політику вибіркового огляду, але автоматичний вибір і межу вартості застосування тут не оцінювали. Сильніша зовнішня перевірка має зафіксувати новий набір реальних дефектів репозиторіїв із незалежними еталонними виправленнями й тестами та звітувати його окремо від сконструйованих тут задач.

Сила доказів. Ефект має однаковий напрям у кожному експерименті, а в об'єднанні всіх виправлених запусків він великий: тихих хибних успіхів стало менше в 12 задачах і більше в жодній, а невдач — менше в 14 і більше у 2 (таблиця 5). Зареєстрований тест на рівні задач лише для кампанії A не досяг значущості (6 краще, 1 гірше, $p = 0{,}125$), бо базова політика вже розв'язала 15 з 25 задач і розбіжних задач було лише сім. Тому головним результатом ми подаємо розмір і послідовність ефекту, а зареєстрований тест — як межу того, наскільки його підтверджено на нових задачах.

8 Обмеження та загрози валідності

- **Добір задач.** Усі задачі — невеликі задачі з виправлення коду на Python, написані автором. Перші 19 задач частково написано *після* виявлення типу збою, і вони надмірно представляють задачі, де метод допомагає. Кампанія A зменшує це упередження (які правила видимі, вирішує жереб), а кампанія B бере баги від локальної моделі, але самі специфікації все ще авторські, з явно сформульованими й пронумерованими правилами, що робить їх легшими за типові docstring реального коду. Розмір ефекту на природних багах невідомий.

- **Повторне використання під час розробки.** Задачі H1–H13 використовувалися під час розробки механізму; R4 і кампанія A використовують нові задачі; кампанія B має нові згенеровані реалізації з помилками для шести специфікацій A, а не нові специфікації.

- **Одиниця аналізу і потужність.** Seed однієї задачі не є незалежними; p-значення на рівні запусків завищують узагальнення на нові задачі. Тести на рівні задач для R4–R5 (таблиця 2) є post hoc; у кампанії A зареєстрований тест на рівні задач не був значущим переважно тому, що базова політика вже розв'язала більшість задач.

- **Причинний висновок і спільні специфікації.** Генерація огляду одночасно змінює модель, подання, інформацію й момент її надання. SPEC не є узгодженим контролем із чеклістом до виправлення. Шість задач кампанії B повторно використовують специфікації A, тому 50 екземплярів не є 50 незалежними специфікаціями; аналіз чутливості на 44 групах є post hoc.

- **Попередня реєстрація.** Реєстрація R4 зберігається в локальному журналі поза контролем версій і не має незалежної часової мітки; кампанії A і B зареєстровано публічними комітами до запуску, з однією задокументованою поправкою до кампанії B (хмарна модель).

- **Еталон.** Приховані перевірки слабші за docstring (підрозділ 6.5); частки верифікованого виконання є верхніми оцінками для обох груп.

- **Межі: одна пара моделей.** Це дослідження однієї конфігурації: Gemma як локальний виконавець і хмарна модель за Antigravity CLI як рецензент. Оголошення Google поєднує Gemma 4 26B A4B з Gemini 3.8 Flash [9]; ми використовуємо меншу `gemma4:12b`, Gemini 4 Argon у кампанії A і Gemini 3.8 Flash у кампанії B. У R1–R5 використовувалася модель CLI за замовчуванням, яку не зафіксовано, тож для цих прогонів не стверджується, що це була Gemini. Щодо інших сімейств моделей жодних тверджень не робиться, а порівняння між кампаніями A і B є описовими.

- **Обладнання.** Один споживчий GPU з частковим вивантаженням на CPU; час виконання чутливий до фонового навантаження.

- **Облік вартості.** У R1–R5 бракує часу хмарної генерації.

- **Перервані прогони.** R5 і кампанії A та B призупинялися на прохання автора й відновлювалися із збережених результатів з тими самими згенерованими тестами й чеклістами, у різних сесіях.

9 Доступність даних і коду

Код і артефакти доступні в `https://github.com/cyberdid/cloud-edge-ai-orchestrator`, гілка `main`. Зафіксований знімок експериментальних даних — ко-

міт `9cd894f717a4235c7cb7bdc36563275ba36bdb0f`; виправлений публікаційний пакет позначено тегом `paper-i-preprint-2026-10-21-rev2`. Контрольні суми та точні інструкції наведено в `docs/paper/README.md`. Із кореня репозиторію команда `bash scripts/paper/reproduce.sh` відтворює підсумки, рисунки й PDF обома мовами зі збережених результатів без викликів моделей. Числові тіла таблиць генерує `docs/paper/tex/gen_tables.py`; додаток Б пов'язує результати з файлами. Архівного DOI ця версія не заявляє.

10 Висновки

Локальний агент-програміст на 12B параметрів може повідомляти про успіх після проходження неповних видимих тестів. У цьому обмеженому дослідженні задач виправлення коду хмарні контрактні тести лише за інтерфейсом зменшили тихі хибні успіхи з 37/150 до 4/150 і підвищили виміряне верифіковане виконання з 66,0% до 87,3% (об'єднання post hoc). Чекліст дав ту саму сукупну частку верифікованого виконання, що й тести в кампанії A, за 11,8% проти 53,7% додаткового часу кампанії з генерацією; еквівалентність не встановлено. Зареєстрований тест на рівні задач не досяг значущості. Результати мотивують подальші дослідження огляду специфікації, залишаючи відкритими причинне пояснення та узагальнення на незалежні реальні дефекти.

Прозорість дослідження

Ліцензія. Текст статті, рисунки й експериментальні записи: Creative Commons Attribution 4.0 International (`https://creativecommons.org/licenses/by/4.0/`); виконуваний код: Apache License 2.0. Межі та сторонні винятки визначено в описі ліцензування репозиторію.
Статус автора та співпраця. Автор є незалежним дослідником і аспірантом Державного торговельно-економічного університету; заявлену співпрацю з кафедрою інженерії програмного забезпечення та кібербезпеки підтверджено автором. Експерименти виконано на обладнанні й облікових записах автора. Співпраця не означає участі чи схвалення Google.

Використання інструментів ШІ. Асистенти ШІ допомагали в розробці програм, реалізації аналізу, підготовці тексту, перекладі та верстці. Досліджувані моделі й налаштування описано окремо. Відгуки ШІ, статичні й числові перевірки не є незалежним людським рецензуванням або незалежною реплікацією експериментів. Автор відповідає за остаточні твердження, джерела й рішення про публікацію.

Література

[1] Nadia Alshahwan, Jubin Chheda, Anastasia Finegenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. Automated unit test improvement using large language models at Meta. In *32nd ACM International Conference on the Foundations of Software Engineering (FSE 2024)*, 2024. arXiv:2402.09171.

[2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5): 507–525, 2015. doi: 10.1109/TSE.2014.2372785. URL `https://discovery.ucl.ac.uk/id/eprint/1471263/`.

[3] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. In *International Conference on Learning Representations (ICLR 2023)*, 2023. arXiv:2207.10397.

[4] Google Research. Harness^2. Software repository, https://github.com/google-research/harness2, 2026.

[5] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. AgentCoder: Multi-agent-based code generation with iterative testing and optimisation, 2024. arXiv:2312.13010v3.

[6] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2310.01798. arXiv:2310.01798v2.

[7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974. arXiv:2403.07974v2.

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2310.06770. arXiv:2310.06770.

[9] Sachin Kotwani and Taylor Mullen. Introducing support for local AI models in the Antigravity SDK. Google Developers Blog, 23 September 2026, https://developers.googleblog.com/introducing-support-for-local-ai-models-in-the-antigravity-sdk/, 2026.

[10] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS 2023)*, 2023. arXiv:2305.01210.

[11] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. URL https://arxiv.org/abs/2303.17651. arXiv:2303.17651v2.

[12] Michele Tufano, James McClure, José Cambronero, Sam Cheng, Sherry Y. Shi, Renyao Wei, Dorothy Chen, Franjo Ivancic, Livio Dalloro, and Pat Rondon. Grounding AI agents in contracts: An empirical evaluation of spec-driven test generation. In *SpecOps '26: Proceedings of the 1st International Workshop on Specification-Driven Development Life Cycle*. ACM, 2026. URL https://research.google/pubs/grounding-ai-agents-in-contracts-an-empirical-evaluation-of-spec-driven-test-generation/.

[13] Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. In *International Conference on Learning Representations (ICLR 2026)*, 2026. arXiv:2510.20270.

A Попередні реєстрації та відхилення

- **Кампанію A** зареєстровано в `docs/paper/PREREGISTRATION_CAMPAIGN_A.md` (коміт `5c9ac2f`, опубліковано 2026-10-20 о 12:52 +03:00) до будь-якого запуску; набір задач

P1–P25 заморожено в `scripts/paper_tasks/manifest.json` (коміт `24ea7a0`). Відхилення: кампанію двічі призупиняли на прохання автора й відновлювали із збережених результатів; скрипт аналізу згодом змінено так, щоб він пропускав категорії без задач (це знадобилося для кампанії B), що не змінює жодного результату кампанії A.

- **Кампанію B** зареєстровано в `docs/paper/PREREGISTRATION_CAMPAIGN_B.md` (коміт `214f0a8`) до генерації; поправка 1 (коміт `60dacd5`, до генерації) змінила хмарну модель на Gemini 3.8 Flash через квоту Gemini 4 Argon і додала зупинку при невдалих викликах хмари. Згенеровані задачі заморожено в `scripts/paper_tasks/natural_tasks.json` (коміт `d2f2807`) до будь-якого запуску виправлення. Залишилося шість задач, менше за зареєстрований поріг у десять, тож кампанію звітовано описово, як і зареєстровано.
- **R4** зареєстровано в локальному журналі (без часової мітки); першу спробу, що впала на зависанні, відкинуто й повторено з виправленим вимірюванням.

Б Карта доказів

Результат	Файли в репозиторії
R1 (підрозділ 6.1)	`benchmarks/spec_contract_experiment.json`; `docs/SPEC_CONTRACT_RESULTS.md`
R2, R3 (підрозділ 6.2)	`benchmarks/spec_contract_control.json`, `benchmarks/spec_contract_v2.json`
R4 (підрозділ 6.3)	`benchmarks/spec_contract_confirm_fresh.json`
R5 (підрозділ 6.4), таблиця 2	`benchmarks/spec_contract_confirm_all.json`; `scripts/analysis/clustered_spec.py`
Якість тестів (підрозділ 6.5)	`benchmarks/oracle_quality.json`; `scripts/analysis/oracle_quality.py`
Негативні результати (підрозділ 6.7)	`benchmarks/gemma_orion_phase2b_escalation_ladder.json`, `benchmarks/harness2_experiment.json`
Кампанія A (підрозділ 6.8)	`benchmarks/campaign_a.json`, `campaign_a_ablation.json`, `campaign_a_analysis.txt`, `campaign_a_test_grading.json`
Кампанія B (підрозділ 6.9)	`scripts/paper_tasks/natural_tasks.json`; `benchmarks/campaign_b.json`, `campaign_b_analysis.txt`, `campaign_b_test_grading.json`
Перевірки під час редагування	`benchmarks/publication_analysis.json`; `scripts/analysis/publication.py`
Об'єднані докази (підрозділ 6.10)	`benchmarks/pooled_analysis.txt`; `scripts/analysis/pooled.py`
Система (розділ 4)	`antigravity_hybrid_orchestrator/engine/spec_tests.py`, `privacy/assist_payload.py`; `scripts/benchmark_gemma_orion_phase2b.py`
Задачі P1–P25	`scripts/paper_tasks/`; `tests/test_paper_tasks.py`
Рисунки	`scripts/analysis/paper_figures.py`, `paper_diagram.py`